```
#include <iostream>
#include <vector>
using namespace std;

const int INF = 999;

void printTables(const vector<vector<int>>& dist,
                 const vector<vector<int>>& nextHop,
                 int n,
                 string title) {

    cout << "\n========================================\n";
    cout << title << "\n";
    cout << "========================================\n";

    for (int i = 0; i < n; i++) {
        cout << "\nRouter " << i + 1 << "\n";
        cout << "Destination\tCost\tNext Hop\n";

        for (int j = 0; j < n; j++) {
            cout << j + 1 << "\t\t" << dist[i][j] << "\t";

            if (nextHop[i][j] == -1)
                cout << "-";
            else
                cout << nextHop[i][j] + 1;

            cout << "\n";
        }
    }
}

int main() {

    int n;

    cout << "Enter number of routers: ";
    cin >> n;

    vector<vector<int>> cost(n, vector<int>(n));

    cout << "\nEnter cost matrix:\n";

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> cost[i][j];
        }
    }
```

```
vector<vector<int>> dist = cost;
vector<vector<int>> nextHop(n, vector<int>(n, -1));

// Initial routing table
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {

        if (i == j) {
            nextHop[i][j] = -1;
        }
        else if (cost[i][j] != INF) {
            nextHop[i][j] = j;
        }
    }
}

printTables(dist, nextHop, n, "INITIAL ROUTING TABLES");

// Distance Vector exchanges
for (int round = 1; round <= 3; round++) {

    vector<vector<int>> newDist = dist;
    vector<vector<int>> newNextHop = nextHop;

    for (int i = 0; i < n; i++) {

        for (int k = 0; k < n; k++) {

            // i and k must be directly connected
            if (cost[i][k] == INF || i == k)
                continue;

            for (int j = 0; j < n; j++) {

                if (dist[k][j] == INF)
                    continue;

                int newCost = cost[i][k] + dist[k][j];

                if (newCost < newDist[i][j]) {
                    newDist[i][j] = newCost;
                    newNextHop[i][j] = k;
                }
            }
        }
    }
```

```
        dist = newDist;
        nextHop = newNextHop;

        if (round == 1)
            printTables(dist, nextHop, n, "FIRST EXCHANGE ROUTING TABLES");

        else if (round == 2)
            printTables(dist, nextHop, n, "SECOND EXCHANGE ROUTING TABLES");

        else
            printTables(dist, nextHop, n, "FINAL ROUTING TABLES");
    }

    // Source and destination
    int source, destination;

    cout << "\nEnter source router: ";
    cin >> source;
    source--;

    cout << "Enter destination router: ";
    cin >> destination;
    destination--;

    cout << "\nShortest Path:\n";

    int current = source;
    cout << current + 1;

    while (current != destination) {
        current = nextHop[current][destination];

        if (current == -1) {
            cout << "\nNo path exists.\n";
            return 0;
        }

        cout << " -> " << current + 1;
    }

    cout << "\nMinimum Cost: "
         << dist[source][destination] << "\n";

    return 0;
}
```